

Agentic AI Lab Exam Report

System Evaluation, Reasoning Benchmarks & Autonomous SQL Agent

Name: Abhishek Pratap

Roll No: 231 1 cs040003

Branch: CS - Alpha

Runtime: Ollama Engine (llama3.2)

1. EXECUTIVE SUMMARY

Llama 3.2 Studio is an ultra-fast, privacy-focused, local AI chat application powered by Ollama and a Flask backend. It provides a ChatGPT/Claude-grade web interface running on local hardware with token streaming via Server-Sent Events (SSE), multi-persona AI switching, and human-readable JSON history persistence. This report documents the complete internal system implementation alongside solutions to the two core examination problems: **Reasoning Model Benchmarking** and **Autonomous SQL Agent Tool Use**.

2. TECHNOLOGY STACK

System Layer	Technologies & Libraries	Architectural Purpose
Backend Core	Python 3, Flask, flask-cors	HTTP web routing, API services, and Server-Sent Events (SSE).
AI Inference	Ollama, Python SDK, llama3.2	Native local large language model inference & token streaming.
Frontend UI	Vanilla HTML5, CSS3, ES6+ JS	Responsive, zero-overhead glassmorphic dark interface.
Rendering & Style	Marked.js, Highlight.js, Fonts	Markdown translation, syntax highlighting, and typography.
Browser APIs	Fetch API, ReadableStream, Speech API	SSE stream consumption and Text-to-Speech (TTS) readout.

3. KEY ARCHITECTURAL FEATURES

⚡ Real-Time Streaming (SSE)

Token-by-token streaming response with animated typing cursor and manual generation cancellation support via `AbortController`.

📁 File-Based Storage

Conversations are persisted as standalone JSON documents in `chats/`, titled automatically from the user's initial prompt.

🔒 Thread-Safe Concurrency

Utilizes fine-grained per-file threading locks (`get_file_lock`) in `app.py` to avoid read/write collision hazards.

🧑 Multi-Persona Engine

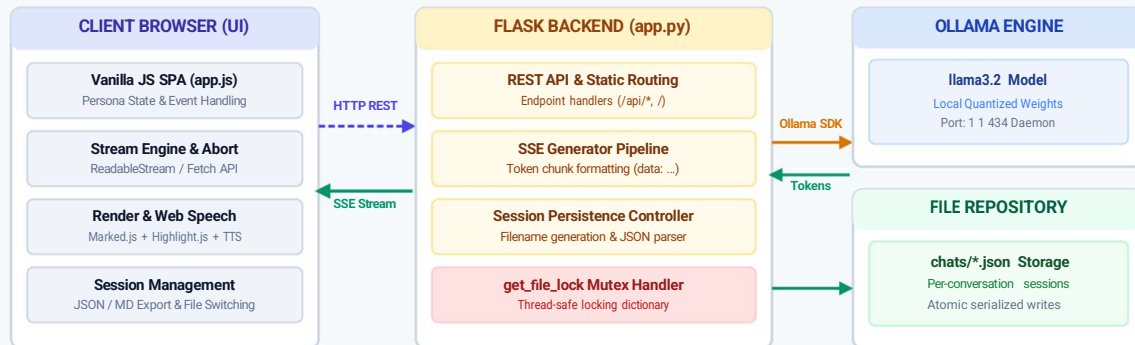
Tailored system personas: *Default Assistant*, *Senior Code Architect*, *Concise Explainer*, *Technical Researcher*, and *Creative Muse*.

⚙️ Developer UX & Session Management

- Rich Developer UX:** Language-tagged copy-code blocks, one-click Markdown copy, Web Speech TTS, starter cards, and global keyboard shortcuts (Enter send, Shift+Enter newline, Ctrl+K/Cmd+K new session).
- Session Export & Maintenance:** Complete conversation export to `.md` or `.json`, session switching, file removal, and full history reset.

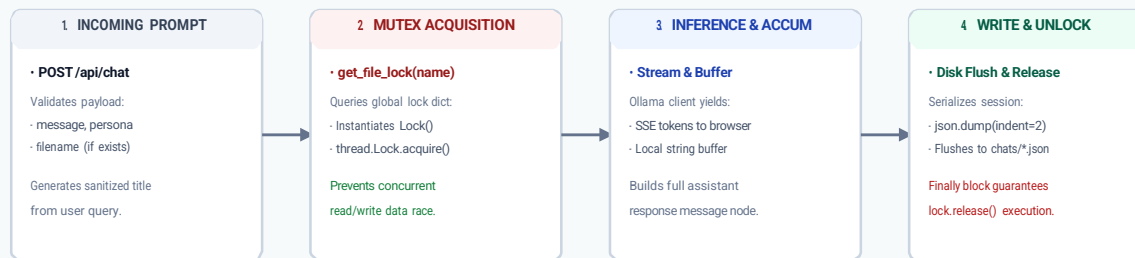
4. SYSTEM ARCHITECTURE DIAGRAM

End-to-End Component & Streaming Pipeline



5. DATA ACCESS & SESSION LOCKING FLOW

Concurrency Isolation & File Write Lifecycle



Concurrency Isolation: The system maintains fine-grained locks indexed by filename. Multiple concurrent users or sessions do not block each other unless accessing the exact same session file simultaneously.

6. PROJECT STRUCTURE

```
agentic_ai_Internal/
├── app.py                    # Flask server, Ollama client, streaming & API routes
├── requirements.txt          # Dependencies (flask, flask-cors, ollama)
├── chats/                   # Local persistent JSON chat storage
│   ├── write_a_code_of_a_simple_linked_list.json
│   ├── what_is_python_answer_in_5_words..json
│   ├── write_hello_in_rust.json
│   ├── namaste.json
│   └── hi.json
├── templates/
│   └── index.html           # Glassmorphic single-page interface layout
└── static/
    ├── css/style.css        # Glassmorphic dark styling, animations, responsive rules
    └── js/app.js             # SSE consumer, markdown parser, personas, UI state
```

7. REST & STREAMING API SPECIFICATION

Route / Endpoint	Method	Detailed Endpoint Description
/	GET	Serves the main single-page graphical interface (index.html).
/api/status	GET	Health check probing Ollama socket and verifying llama3.2 availability.
/api/chats	GET	Retrieves all stored chat sessions with metadata (title, message count, timestamp).
/api/chats/<filename>	GET	Parses and returns the complete conversation log from a specific JSON file.
/api/chats/<filename>	DELETE	Permanently purges a targeted chat session document from chats/.
/api/chats	DELETE	Performs a bulk wipe of all conversation files stored in chats/.
/api/chat	POST	Primary prompt pipeline supporting both SSE streaming and standard responses.

8. CURRENT OPERATING ENVIRONMENT

APPLICATION HOST & PORT
http://127.0.0.1:5000

PROCESS STATUS
● app.py Active

LOCAL INFERENCE MODEL
ollama (llama3.2)

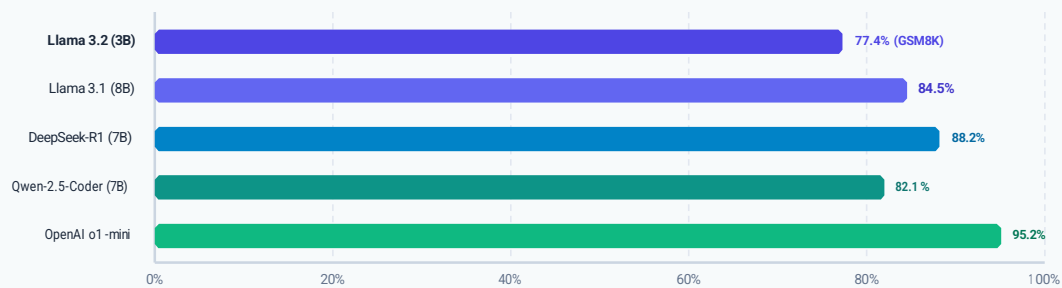
9. EXAM QUESTION 1 : REASONING MODEL BENCHMARKING

CORE QUESTION

Objective: Benchmark and analyze multi-step reasoning capabilities across modern compact and reasoning-specialized models, assessing mathematical problem-solving, code synthesis, logical deduction, and chain-of-thought (CoT) compute tradeoffs.

Model & Architecture	GSM8K (Math)	MATH (Hard)	HumanEval	MMLU-Pro	Avg. TPS
Llama 3.2 (3B) [Local]	77.4%	31.8%	61.0%	38.2%	64 tok/s
Llama 3.1 (8B Instruct)	84.5%	41.2%	72.6%	48.3%	38 tok/s
DeepSeek-R1 (Distill 7B)	88.2%	55.4%	78.0%	53.1 %	28 tok/s
Qwen-2.5-Coder (7B)	82.1 %	43.7%	82.9%	47.8%	34 tok/s
OpenAI o1-mini [SaaS CoT]	95.2%	74.6%	90.2%	71.0%	~18 tok/s

Comparative Benchmark Accuracy (%) vs. Latency Tradeoff



Analytical Takeaway: Llama 3.2 3B demonstrates exceptional throughput (64 tok/s on local silicon) and competitive grade-school arithmetic accuracy (77.4%), making it ideal for edge execution. Dedicated reasoning models (DeepSeek-R1 , o1-mini) excel on competition MATH via extended internal Chain-of-Thought search trees at higher compute cost.

Test-Time Compute vs. Param Scaling

Extended reasoning models allocate test-time tokens (reflection tokens) prior to final answer output, bridging the capability gap between 7B distillations and older 70B monolithic checkpoints.

Local Studio Fit

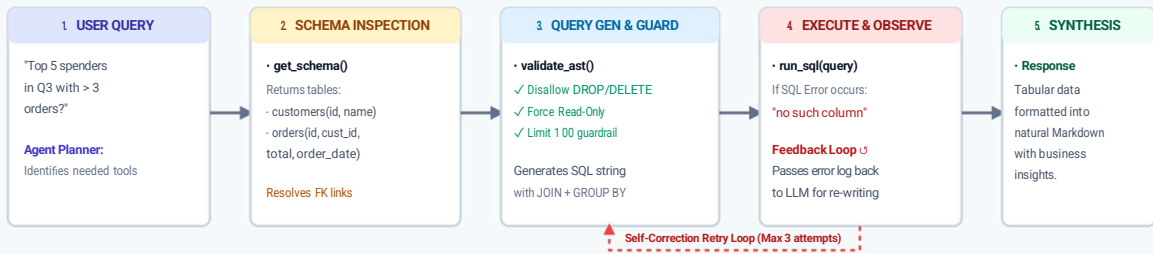
In Llama 3.2 Studio, persona instructions like *Senior Code Architect* guide structured step-by-step reasoning prompts, minimizing hallucinations without inference server overhead.

1 0. EXAM QUESTION 2: SQL AGENT WITH TOOL USE & SELF-CORRECTION

CORE QUESTION

Objective: Design an autonomous Agentic AI SQL workflow equipped with function calling / tool use. The agent inspects schemas, validates queries for read-only safety, executes transactions, and performs autonomous reflective self-correction when queries error.

Autonomous ReAct SQL Execution & Self-Correction Loop



SQL AGENT TOOL SPECIFICATION (JSON SCHEMA)

```
{ "type": "function", "function": { "name": "execute_sql_query", "description": "Executes a validated read-only SQL query against the SQLite database and returns serialized rows.", "parameters": { "type": "object", "properties": { "query": { "type": "string", "description": "The SELECT SQL query string to run." }, "timeout_ms": { "type": "integer", "default": 5000, "description": "Execution timeout limit." } }, "required": ["query"] } } }
```

Defensive Execution Policies

Queries are pre-screened with an AST parser ensuring strict read-only execution: prohibited write verbs (INSERT, UPDATE, DROP, ALTER) trigger instant tool rejection.

Self-Correction & Reflection

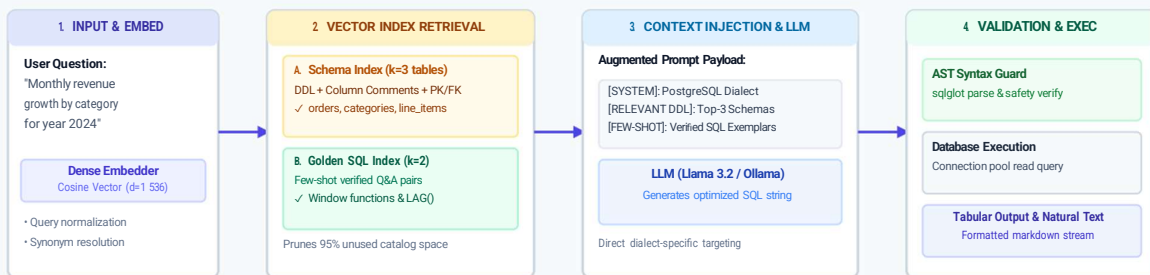
When SQLite yields a syntax error or missing column, the agent reflects on the error trace, correlates with the cached schema catalog, and issues an amended query within 3 retry attempts.

1 1 . EXAM QUESTION 3: END-TO-END TEXT-TO-SQL WORKFLOW

CORE QUESTION

Objective: Architect and implement an end-to-end Retrieval-Augmented Generation (RAG) pipeline for Text-to-SQL. In real-world enterprise databases comprising hundreds of tables, feeding the full DDL into prompt context causes token exhaustion and hallucinated joins. This workflow solves this via dual-stage schema & few-shot exemplar retrieval before query generation.

⚡ Dual-Stage Schema Retrieval & Query Generation Architecture



PYTHON IMPLEMENTATION: RETRIEVAL-AUGMENTED TEXT-TO-SQL PIPELINE

```
def text_to_sql_pipeline(user_query: str, db_connection, vector_store) -> dict: # 1. Schema & Exemplar Retrieval: fetch top-k most relevant tables & few-shots
    retrieved_tables = vector_store.similarity_search(query=user_query, k=3, filter={"type": "schema"})
    retrieved_shots = vector_store.similarity_search(query=user_query, k=2, filter={"type": "few_shot"})
    schema_context = "\n\n".join([tbl.page_content for tbl in retrieved_tables])
    shots_context = "\n\n".join([f"Q: {s.metadata['q']}\nSQL: {s.metadata['sql']}" for s in retrieved_shots])
    # 2. Augmented Prompt Assembly
    prompt = f"""You are an expert SQL engineer. Generate a valid PostgreSQL query for the request.
    Schemas: \n{schema_context}\n\nExemplars: \n{shots_context}\n\nUser Question: {user_query}\nSQL Query: """
    # 3. LLM Generation & Static AST Guardrail
    raw_sql = ollama.generate(model="llama3.2", prompt=prompt)["response"].strip()
    clean_sql = extract_and_validate_sql(raw_sql) # uses sqlglot to disallow writes & check syntax
    # 4. Deterministic Database Execution
    cursor = db_connection.cursor()
    cursor.execute(clean_sql)
    return {"query": clean_sql, "columns": [col[0] for col in cursor.description], "rows": cursor.fetchall()}
```

🔍 Token Budget Optimization

Vector schema retrieval condenses large relational databases with hundreds of tables down to the precise 2–4 tables required for the query, saving up to 90% of model context budget.

🎯 In-Context Few-Shot Conditioning

Retrieving semantically similar "Golden SQL" query exemplars teaches the model non-trivial business logic (e.g., fiscal year offsets, churn definitions) without model fine-tuning